Hooks, Fixtures and Test Organisation

COURSE SPECIFICATIONS

• Track: Playwright Automation with JavaScript + AI • Level: Intermediate to Advanced
• Focus: Test Suites, test.extend Fixtures, Config • Prerequisites: Days 1–7
• Architecture: Dependency Injection with await use() • Test Runner: @playwright/test (.js)

Architect maintainable, scalable test suites using test lifecycle hooks (beforeEach/afterEach), built-in and custom fixtures (test.extend), modular helper utilities, and production playwright.config.js configurations.

Day 8 Learning Objectives

- Structure clean, readable test suites using test.describe() grouping blocks and feature-based directory layouts.
- Implement test lifecycle hooks: test.beforeEach(), test.afterEach(), test.beforeAll(), and test.afterAll().
- Differentiate between per-test setup and shared setup, and avoid shared mutable state anti-patterns in parallel runs.
- Understand Playwright fixtures architecture (built-in page, context, browser) and fixture lifecycle management.
- Author custom domain fixtures using test.extend() to inject pre-authenticated sessions or mock test data.
- Design reusable helper functions that pass locators and test data without creating rigid, brittle abstractions.
- Configure playwright.config.js (testDir, fullyParallel, workers, retries, reporters, use) and execute targeted CLI commands.

As your test automation repository scales from 5 tests to 500 tests, organizing files by business capability rather than technical arbitrary layers is essential.

Grouping Tests with `test.describe`

`test.describe()` groups related tests together into logical blocks. It clarifies reports and allows sharing setup hooks across specific subsets of tests:

```javascript
// tests/customer/customer_management.spec.js
const { test, expect } = require('@playwright/test');

test.describe('Customer Management Module', () => {

  test.describe('Customer Creation Flow', () => {
    test('creates a standard enterprise customer profile', async ({ page }) => {
      // ...
    });

    test('validates tax identifier format', async ({ page }) => {
      // ...
    });
  });

  test.describe('Customer Search & Filtering', () => {
    test('filters customers by subscription status', async ({ page }) => {
      // ...
    });
  });

});
```

Recommended Project Directory Structure

```text
my-playwright-project/
% % %  playwright.config.js          # Central configuration
% % %  package.json
% % %  fixtures/                    # Test data (JSON, CSV, PDF attachments)
%    % % %  sample_customer.json
%    % % %  test_upload.pdf
% % %  helpers/                     # Small reusable helper utilities
%    % % %  authHelper.js
%    % % %  customerHelper.js
% % %  tests/                       # Feature-organized test files
   % % %  auth/
   %    % % %  login.spec.js
```

```
%    % % %  password_reset.spec.js
% % %  customer/
%    % % %  customer_creation.spec.js
%    % % %  customer_search.spec.js
% % %  billing/
    % % %  invoice_download.spec.js
```

When to Split Large Test Files

- **Rule of Thumb: If a test file exceeds 200 lines or covers multiple distinct business user journeys, split it into separate `.spec.js` files.**

- Separate files run concurrently across worker threads, drastically reducing overall CI execution duration!

Clean Describe Grouping Pattern (A structured file with nested describe blocks demonstrating clear test hierarchy.)

```
test.describe('Workspace Settings', () => {
  test.describe('Member Invitations', () => {
    test('sends invitation email with admin privileges', async ({ page }) => {
      // ...
    });
    test('blocks invitations with invalid domain', async ({ page }) => {
      // ...
    });
  });
});
```

Lifecycle hooks allow you to execute setup and teardown code before and after tests run.

The 4 Playwright Lifecycle Hooks

1. **`test.beforeEach(async ({ page }) => { ... })`**
   - Runs **once before each individual test** in the file or describe block.
   - Ideal for: Navigating to the starting page, resetting form state, or setting up test-specific preconditions.
2. **`test.afterEach(async ({ page }) => { ... })`**
   - Runs **once after each individual test**.
   - Ideal for: Cleaning up temporary test records, clearing local storage, or capturing custom logs.
3. **`test.beforeAll(async () => { ... })`**
   - Runs **once before all tests** in the file start.
   - Ideal for: Heavy one-time setup (e.g. creating a test database schema, seeding master data via API).
   - *Note:* Does not receive the `page` fixture by default because pages are isolated per test!
4. **`test.afterAll(async () => { ... })`**
   - Runs **once after all tests** in the file complete.
   - Ideal for: Teardown (e.g. purging database test accounts, closing database connections).

Critical Warning: Shared Mutable State Anti-Pattern!

```javascript
// 'L DANGEROUS ANTI-PATTERN: Shared mutable state across tests!
let customerId;

test.beforeAll(async () => {
  // Creating shared mutable variable
  customerId = 'CUST-1001';
});

test('edits customer name', async ({ page }) => {
  // Modifies shared customer
});

test('deletes customer', async ({ page }) => {
  // If this runs before 'edits customer', the other test fails!
});
```

Rule: Tests must be **independent**. Use `beforeEach` for isolated test-level setup, and generate unique IDs for each test (e.g. `customer-${Date.now()}`).

Standard beforeEach Pattern (Navigating and verifying landing page before every test in a suite.)

```
test.describe('Customer Dashboard', () => {
  test.beforeEach(async ({ page }) => {
    // Isolated setup for every test:
    await page.goto('/customers');
    await expect(page.getByRole('heading', { name: 'Customer Directory' })).toBeVisible();
  });

  test('searches by email address', async ({ page }) => {
    await page.getByPlaceholder('Search customers').fill('alex@rootaccess.org');
    await expect(page.getByRole('row')).toHaveCount(2); // header + 1 result
  });

  test('filters by active status', async ({ page }) => {
    await page.getByLabel('Status Filter').selectOption('active');
    await expect(page.getByTestId('badge-active')).toBeVisible();
  });
});
```

Fixtures are one of Playwright's most powerful architectural features. If you have ever wondered where `{ page }` or `{ context }` in your test parameters comes from, they are **built-in fixtures**!

What Is a Fixture?

A fixture is an isolated environment component that is prepared before your test runs and cleaned up automatically after your test finishes.

Key advantages of fixtures over `beforeEach` / `afterEach`:

1. **On-Demand:** Fixtures only run if your test explicitly requests them in its arguments.
2. **Encapsulation:** Setup and teardown live together in a single function using the `use()` callback.
3. **Composability:** Fixtures can depend on other fixtures.

How a Fixture Works Under the Hood

```javascript
// Simplified mental model of the built-in { page } fixture:
async function pageFixture({ context }, use) {
  // 1. SETUP: Create a new page
  const page = await context.newPage();

  // 2. RUN TEST: Hand the page over to the test
  await use(page);

  // 3. TEARDOWN: Automatically closes page when test finishes!
  await page.close();
}
```

Creating a Custom Fixture with `test.extend()`

You can define your own domain fixtures (such as an authenticated admin page or pre-seeded customer data):

```javascript
// fixtures/customerFixture.js
const base = require('@playwright/test');

// Extend base test with custom fixture
const test = base.test.extend({
  // Define custom fixture:
  customerData: async ({}, use) => {
    // Setup: Generate unique customer object
    const customer = {
      name: `Test Client ${Date.now()}`,
      email: `client_${Date.now()}@rootaccess.org`,
      plan: 'Enterprise',
    };

    // Provide to test:
    await use(customer);

    // Teardown (if needed, e.g. API cleanup):
    console.log(`Finished test for ${customer.email}`);
  },
});
```

module.exports = { test, expect: base.expect };
```

Using the custom fixture in a test:
```javascript
const { test, expect } = require('./fixtures/customerFixture');

test('creates customer using custom fixture', async ({ page, customerData }) => {
  await page.goto('/customers/new');
  await page.getByLabel('Client Name').fill(customerData.name);
  await page.getByLabel('Email').fill(customerData.email);
  await page.getByRole('button', { name: 'Save' }).click();

  await expect(page.getByRole('alert')).toContainText(customerData.name);
});
```

Custom Authenticated Page Fixture (A custom fixture that automatically logs in before yielding the page.)

```
const base = require('@playwright/test');

const test = base.test.extend({
  authenticatedPage: async ({ page }, use) => {
    // 1. Setup login
    await page.goto('/login');
    await page.getByLabel('Email').fill('admin@rootaccess.org');
    await page.getByLabel('Password').fill('AdminPass123!');
    await page.getByRole('button', { name: 'Sign In' }).click();
    await base.expect(page).toHaveURL(/.*dashboard/);

    // 2. Yield to test
    await use(page);
  },
});
```

While fixtures manage environment state, **helper functions** encapsulate repeated UI workflows into concise, reusable utility functions.

When to Use a Helper Function
If you find yourself copy-pasting the same 4 lines of form inputs across 10 tests, extract them into a clean helper:
```javascript
// helpers/customerHelpers.js

/**
 * Fills out and submits the customer creation form.
 * @param {import('@playwright/test').Page} page
 * @param {Object} customer
 */
async function createCustomer(page, customer) {
  await page.getByLabel('Customer Name').fill(customer.name);
  await page.getByLabel('Contact Email').fill(customer.email);
  await page.getByLabel('Subscription Plan').selectOption(customer.plan);
  await page.getByRole('button', { name: 'Save Customer' }).click();
}

module.exports = { createCustomer };
```

Rules for Writing Clean Helpers
1. **Pass the `page` object explicitly: Helpers should be pure functions that accept `page` as their first argument.**
2. **Avoid Global State: Helpers must not store internal variables or state across calls.**
3. **Keep Assertions Close to the Test: Helpers should execute the steps; the test itself should assert the expected business outcome. This keeps test failures readable and points directly to the failed test assertion.**
```

Using Helpers in a Clean Spec File (A test utilizing a reusable helper function for repetitive data entry.)

```
const { test, expect } = require('@playwright/test');
const { createCustomer } = require('../../helpers/customerHelpers');

test('creates a standard enterprise customer', async ({ page }) => {
  await page.goto('/customers/new');

  // Use helper
  await createCustomer(page, {
    name: 'Acme Corporation',
    email: 'billing@acme.com',
    plan: 'enterprise',
  });

  // Assert outcome in the test
  await expect(page.getByRole('alert')).toHaveText('Customer created successfully');
});
```

`playwright.config.js` is the control room of your automation framework.

Production Configuration Anatomy

```javascript
// playwright.config.js
const { defineConfig, devices } = require('@playwright/test');

module.exports = defineConfig({
  // Directory where Playwright searches for test files
  testDir: './tests',

  // Run tests in files in parallel
  fullyParallel: true,

  // Fail build on CI if test.only is accidentally committed
  forbidOnly: !!process.env.CI,

  // Retry on CI only
  retries: process.env.CI ? 2 : 0,

  // Opt out of parallel tests on CI if server resources are limited
  workers: process.env.CI ? 2 : undefined,

  // Reporter to use
  reporter: [
    ['html', { open: 'never' }],
    ['list'],
  ],

  // Shared settings for all projects
  use: {
    baseURL: 'http://localhost:3000',
    trace: 'on-first-retry',
    screenshot: 'only-on-failure',
    video: 'retain-on-failure',
  },

  // Cross-browser project matrix
  projects: [
    {
      name: 'chromium',
      use: { ...devices['Desktop Chrome'] },
    },
    {
      name: 'firefox',
      use: { ...devices['Desktop Firefox'] },
    },
    {
      name: 'webkit',
      use: { ...devices['Desktop Safari'] },
    },
  ],
```

});
```

### Essential CLI Execution Commands

| Goal | Command |
| :--- | :--- |
| **Run all tests** | `npx playwright test` |
| **Run specific file** | `npx playwright test tests/customer.spec.js` |
| **Run tests matching name** | `npx playwright test -g "should create customer"` |
| **Run in headed mode** | `npx playwright test --headed` |
| **Run on specific browser** | `npx playwright test --project=firefox` |
| **Interactive UI Runner** | `npx playwright test --ui` |
| **View HTML report** | `npx playwright show-report` |

**Targeted CLI Run Examples (Common command-line invocations for daily QA workflow.)**

```
// Run customer tests only on Chromium with UI runner:
npx playwright test tests/customer/ --project=chromium --ui

// Run specific smoke tag:
npx playwright test -g "@smoke"
```

In this practical lab, you will refactor a monolithic, messy customer management test file into a maintainable, enterprise-grade Playwright suite using hooks, fixtures, and helpers.

The Monolithic Problem Code

The existing test file has:
- Repeated `page.goto('/customers')` copy-pasted in 5 places.
- Duplicated form filling code.
- Hardcoded customer names that collide when tests run in parallel.
- No hooks or fixtures.

Lab Refactoring Goals:

1. **Describe Blocks: Group tests cleanly with `test.describe()`.**
2. **Lifecycle Hooks: Use `test.beforeEach()` to navigate to `/customers` and verify page readiness.**
3. **Custom Fixture: Implement a custom fixture with `test.extend()` that generates a unique customer test data object.**
4. **Reusable Helper: Extract a pure `createCustomer(page, data)` helper function.**
5. **Configuration: Provide a clean `playwright.config.js` snippet with `fullyParallel: true` and proper timeouts.**
6. **Independence: Ensure every test passes without depending on the execution of previous tests.**

Refactored Suite Preview (A glimpse of the refactored, modular test structure.)

```
const { test, expect } = require('./customerFixture');
const { createCustomer } = require('./customerHelper');

test.describe('Customer Management Module', () => {
  test.beforeEach(async ({ page }) => {
    await page.goto('/customers');
  });

  test('creates customer cleanly', async ({ page, newCustomer }) => {
    await createCustomer(page, newCustomer);
    await expect(page.getByRole('alert')).toContainText(newCustomer.name);
  });
});
```
```

lifecycle hooks with automatic teardown (await use), and multi-worker isolation.

Task: Submit tests/day8-fixtures.spec.js with complete terminal logs and technical architectural rationale.

Assignment Starter Code (tests/day8-fixtures.spec.js)

```
// tests/customer_suite.spec.js
// DAY 8 ASSIGNMENT: Hooks, Fixtures and Test Organisation
// Refactor this customer management suite using test.describe(),
// beforeEach hooks, a custom fixture with test.extend(), and reusable helpers.

const base = require('@playwright/test');
const { expect } = base;

// 1. Define custom fixture with test.extend()
const test = base.test.extend({
  // Custom fixture that generates isolated, dynamic customer test data
  newCustomer: async ({}, use) => {
    const uniqueId = Date.now();
    const customer = {
      name: `Apex Enterprise ${uniqueId}`,
      email: `contact_${uniqueId}@apexcorp.io`,
      plan: 'Enterprise',
      tierCode: 'ENT-99',
    };

    // Yield to test
```

10 Rubric Requirements for Day 8 Assignment:

1. Pure JavaScript structure using test and expect from @playwright/test
2. Groups related tests logically using test.describe()
3. Leverages beforeEach / afterEach lifecycle hooks
4. Defines custom fixture using test.extend()
5. Yields prepared fixture resource using await use(...)
6. Executes deterministic teardown after the use() invocation
7. Contains at least 2 distinct test() blocks consuming the custom fixture
8. Utilizes web-first assertions for verification
9. Complete terminal test execution log showing passed status
10. Comprehensive technical explanation of fixture dependency injection and worker vs test scope isolation

Q1. What is a Playwright fixture, and how does it differ from traditional beforeEach / afterEach hooks?

[Difficulty: Intermediate] Evaluation Focus: Understanding fixture architecture, encapsulation, and lifecycle management.

Suggested Answer: A fixture in Playwright is an on-demand dependency with an encapsulated setup and teardown lifecycle. Unlike beforeEach hooks that run globally for every test regardless of need, a fixture only executes if a test explicitly requests it in its arguments ({ myFixture }). Furthermore, setup and teardown are co-located in a single function using the await use(value) pattern, which prevents memory leaks and makes fixtures highly composable and reusable across multiple test files.

Follow-Up Question: "How does a fixture clean up its resources after a test completes or fails?"

Q2. Why should automated tests avoid sharing mutable state, and how does parallel execution highlight this issue?

[Difficulty: Intermediate] Evaluation Focus: Test independence, parallel execution, and flaky test elimination.

Suggested Answer: When tests share mutable state (such as Test 1 creating an account and Test 2 editing it via a shared global variable), the suite is coupled to sequential execution. Playwright executes tests concurrently across multiple worker processes by default. In parallel execution, Test 2 may run before Test 1 completes, or they may execute on different worker threads that share no memory, causing intermittent failures. Ensuring tests are strictly independent guarantees deterministic execution in any order or environment.

Follow-Up Question: "What strategies can you use to ensure each test operates on unique data?"

Q3. Explain the purpose of forbidOnly: !!process.env.CI in playwright.config.js.

[Difficulty: Intermediate] Evaluation Focus: Continuous integration quality engineering and pipeline safeguards.

Suggested Answer: Developers frequently use test.only() during local development to focus on a single failing test. If test.only is accidentally committed to source control and pushed to CI, Playwright would run only that single test and skip the rest of the entire regression suite, giving a false sense of security. forbidOnly instructs Playwright to fail the CI build immediately if test.only is detected, preventing broken code from passing CI unverified.

Follow-Up Question: "What other CI-specific configurations should you set in playwright.config.js?"

Q4. You have 10 test files that all require an authenticated admin session. How would you design this efficiently using Playwright fixtures?

[Difficulty: Scenario-Based] Evaluation Focus: Scalable authentication architecture and test.extend().

Suggested Answer: Using test.extend(), create a custom fixture called adminPage. Inside the fixture, initialize a clean page, navigate to the login route or inject authentication state using storageState, and assert that the user is on the admin dashboard. Then call await use(page). Any test that declares { adminPage } receives a pre-authenticated session instantly without duplicating login steps in 10 separate test files.

Follow-Up Question: "How does Playwright storageState improve authentication speed over UI login?"

Q5. What is the recommended design pattern for writing reusable helper functions in Playwright?

[Difficulty: Intermediate] Evaluation Focus: Modular test design and maintainability.

Suggested Answer: Reusable helpers should be pure asynchronous functions that accept the page object as their first parameter, followed by any input parameters (e.g. async function fillCustomerForm(page, customerData)). They should encapsulate repeated interaction steps (fill, select, click) but avoid embedding business assertions, which should remain in the test spec. This keeps the helper focused on execution and preserves clear test reporting.

Follow-Up Question: "When would you transition from helper functions to a formal Page Object Model class?"

1. Which Playwright method is used to define custom test fixtures with encapsulated setup and teardown?

Key Concept: test.extend() is the official API used to extend the base Playwright test object with custom fixtures.

2. In a custom fixture, where does teardown code execute relative to await use(fixtureValue)?

Key Concept: The use() callback yields to the test function. Once the test concludes, execution resumes immediately after use(), running the teardown logic.

3. What is the primary benefit of test.describe() grouping blocks in Playwright?

Key Concept: test.describe() groups related tests, structures HTML reports, and limits the scope of lifecycle hooks to the tests within that block.

4. What does the forbidOnly: !!process.env.CI configuration setting do?

Key Concept: forbidOnly causes Playwright to fail the run if test.only() is present, preventing accidental skipping of the regression suite in CI.

5. Which lifecycle hook runs once before EVERY individual test in a describe block?

Key Concept: test.beforeEach() executes prior to every single test in its enclosing file or describe scope.

6. Why does test.beforeAll() NOT receive the { page } fixture by default?

Key Concept: To prevent cross-test contamination and enable parallelism, Playwright allocates a fresh BrowserContext and Page per test. Therefore, a page cannot be shared across tests in beforeAll.

7. Which CLI command executes tests matching the title "should create customer" exclusively?

Key Concept: The -g (or --grep) flag filters test execution by matching test titles against a string or regular expression.

8. What is the key advantage of fixtures over global beforeEach hooks?

Key Concept: Fixtures are lazy and modular. If a test does not require a database connection or authentication, that fixture simply never runs for that test.

9. What is the purpose of passing the page object as the first parameter to a reusable helper function?

Key Concept: Passing page explicitly keeps helpers as pure functions that operate on the caller's isolated page instance, eliminating global mutable state.

10. What setting in playwright.config.js instructs Playwright to execute all tests across all files concurrently?

Key Concept: fullyParallel: true enables parallel execution of both files and individual tests within those files across available worker threads.